2026

Development Stage Portfolio

Stanislav Starishko

Equipment and Resource Booking Management System

Computing: Software Development: Graded Unit 2

Contents

Introduction 2

Development Roadmap 2

Application Development 3

- **Project Setup** 3
- **Problem Domain Implementation** 3
  - *Entities and Enums* 3
  - *Repositories* 4
  - *Service Layer and Business Rules* 4
  - *Custom Exceptions* 5
- **User Interface Implementation** 5
- **Unfamiliar Technologies and Constructs** 7
- **Error Handling and Validation** 7
  - *Input Validation* 7
  - *Global Exception Handler* 7
  - *Application Logging* 7
- **Internal Documentation** 8
  - *Naming conventions* 8
  - *Package structure* 9
  - *Code comments* 9
  - *Indentation and formatting* 9

Testing 10

- **Test Plan** 10
  - *Testing Approach* 10
  - *Tools* 11
  - *Test Cases* 11
  - *Manual Test Scenarios* 12
- **Test Run 1: Automated Tests** 13
- **Manual Testing** 14
- **Bug Fixes and Regression Testing** 15
- **Test Run 2: After Bug Fix** 17

User Documentation 18

- **Approach** 18
- **Help Page** 18
- **Contextual Help** 18
- **FAQ Section** 19
- **Technical Documentation (Javadoc)** 19

Conclusion 20

Introduction

This document is the Development Stage Portfolio for my Graded Unit 2 project. It covers the full implementation process: from project setup and core development through to testing, bug resolution and user documentation. The project itself is an **Equipment and Resource Booking Management System**: a web application that allows a small organisation to register equipment, manage bookings, track availability and enforce business rules around reservations.

The Planning Stage produced a detailed Action Plan Report with refined requirements, a domain model, database design, UML diagrams and wireframes. This portfolio picks up where that report left off. Rather than repeating the design rationale, I focus here on how the design was translated into working code, what decisions were made during implementation, and how the finished application was tested and documented.

To structure the work, I broke the Development Stage into eight phases, each with a clear scope and deliverable. This phased approach gave me a predictable workflow and made it easier to track progress against the project schedule defined in MS Project. The roadmap below shows the full sequence of each phase.

Development Roadmap

The table below outlines the phases I planned at the start of the Development Stage. Each phase builds on the previous one, following a bottom-up approach: data layer first, then business logic, then user interface, then error handling, and finally testing and documentation.

One point worth noting: **Phase 7** was originally titled "User Documentation & Help". During testing (Phase 6), I realised that writing a separate PDF manual would be less useful than building help features directly into the application. I renamed this phase to **UI Refinement and User Help** to reflect that decision. This kind of adjustment is normal in iterative development; the scope did not change, only the delivery approach.

Phase	Title	Scope
1	**Project Setup**	Spring Boot project, Maven, Git, directory structure
2	**Data Layer**	JPA entities, enums, repositories, data initialisation
3	**Business Logic**	Service layer, business rules BR-01 to BR-08, custom exceptions
4	**UI Layer**	Controllers, Thymeleaf templates, CSS, navigation
5	**Error Handling**	Global exception handler, Bean Validation, logging
6	**Testing**	Test plan, unit tests, integration tests, manual testing
7	**UI Refinement & User Help**	Built-in help page, contextual guidance, FAQ
8	**Code Review & Documentation**	Javadoc, naming review, final portfolio preparation

Application Development

This section documents the implementation of the application, organised by the development phases outlined in the roadmap. Each subsection covers what was built, how it maps to the original design, and any decisions made along the way.

Project Setup

The project was generated using **Spring Initializr** (start.spring.io) with the following configuration: Spring Boot 4.0.5, Java 25, Maven build system, and the dependencies listed below. The generated archive was imported into IntelliJ IDEA and pushed to a GitHub repository for version control.

Selected dependencies and their purpose:

Dependency	Purpose
spring-boot-starter-data-jpa	JPA entity management, Spring Data repositories, Hibernate ORM
spring-boot-starter-webmvc	MVC web framework, embedded Tomcat, request routing
spring-boot-starter-thymeleaf	Server-side HTML templating engine
spring-boot-starter-validation	Bean Validation (JSR 380) with Hibernate Validator
h2 (runtime)	Embedded in-memory SQL database for development and testing
lombok	Compile-time code generation for getters, setters, constructors

The project follows a standard **layered architecture**: model (entities and enums), repository (data access), service (business logic), controller (request handling), and templates (views). This structure maps directly to the MVC pattern described in the Planning Stage under NFR-04.

Problem Domain Implementation

The problem domain consists of four JPA entities, three enums, four Spring Data repositories, and a service layer that enforces all eight business rules defined in the Planning Stage.

Entities and Enums

The entity classes follow the domain model from the Planning Stage. Each entity is annotated with @Entity and mapped to a database table using JPA annotations. Lombok provides boilerplate reduction through @Getter, @Setter and @NoArgsConstructor, while business methods such as hasConflictWith(), isAvailable() and getDurationDays() are implemented manually.

Entity	Table	Key Design Points
Category	category	Unique name constraint. EAGER-loaded equipment list for category page display.
User	app_user	Table named app_user to avoid SQL reserved word conflict with "user". Stores role as VARCHAR via @Enumerated(STRING).
Equipment	equipment	ManyToOne to Category (BR-04). Status stored as enum string. Cost uses BigDecimal to avoid rounding.

Entity	Table	Key Design Points
Booking	booking	ManyToOne to Equipment and User. @PrePersist sets createdAt timestamp automatically. Domain methods for conflict detection and duration.

Three enums define the constrained status values: **EquipmentStatus** (AVAILABLE, BOOKED, MAINTENANCE, UNAVAILABLE), **BookingStatus** (CONFIRMED, CANCELLED), and **UserRole** (ADMIN, STAFF). These are stored as VARCHAR columns via JPA, which keeps the schema simple without losing type safety at the application level.

Repositories

Each entity has a corresponding Spring Data JPA repository that extends JpaRepository. This provides standard CRUD operations without any manual implementation. Custom query methods are added where the business logic requires them.

The most significant custom query is findConflictingBookings() in BookingRepository. This uses a JPQL query to find confirmed bookings for a given equipment item that overlap with a specified date range. It directly supports **BR-02** (no overlapping bookings) and is the only repository method with hand-written query logic; everything else uses Spring Data derived queries such as findByCategoryId() or findByStatus().

Service Layer and Business Rules

The service layer sits between the controllers and repositories. Each service is annotated with @Service and uses constructor injection for its dependencies. The table below shows how the eight business rules from the Planning Stage are implemented.

Rule	Description	Enforced By	Mechanism
BR-01	Cannot book equipment in maintenance or unavailable	EquipmentService	verifyAvailableForBooking() throws EquipmentNotAvailableException
BR-02	No overlapping bookings for same item	BookingService	JPQL query findConflictingBookings() via BookingRepository
BR-03	Every booking linked to a registered user	BookingService	userService.findById() before saving
BR-04	Every equipment item belongs to one category	JPA constraint	@ManyToOne with nullable=false on category_id
BR-05	End date must be equal to or later than start date	BookingService	Date comparison before any other checks
BR-06	Cannot delete category with assigned equipment	CategoryService	Checks findByCategoryId() before delete
BR-07	Cannot delete equipment with active bookings	EquipmentService	Filters bookings by CONFIRMED status before delete
BR-08	Equipment status reflects active bookings	BookingService	Calls equipmentService.updateStatus() after booking creation

An important architectural decision: **BookingService calls EquipmentService** for availability checks and status updates rather than accessing EquipmentRepository directly. This keeps each service responsible for its own entity and avoids duplicating logic. The sequence diagram in the Planning Stage shows this exact interaction pattern.

Custom Exceptions

Five custom exception classes handle specific business rule violations. All extend RuntimeException so they propagate through the Spring transaction boundary without requiring checked exception declarations.

Exception	Rules	When Thrown
ResourceNotFoundException	All	Entity not found by ID in any service
BookingConflictException	BR-02	Overlapping confirmed booking detected
EquipmentNotAvailableException	BR-01	Booking attempted on maintenance/unavailable item
InvalidBookingDatesException	BR-05	End date before start date
DeletionBlockedException	BR-06, BR-07	Delete blocked by dependent records

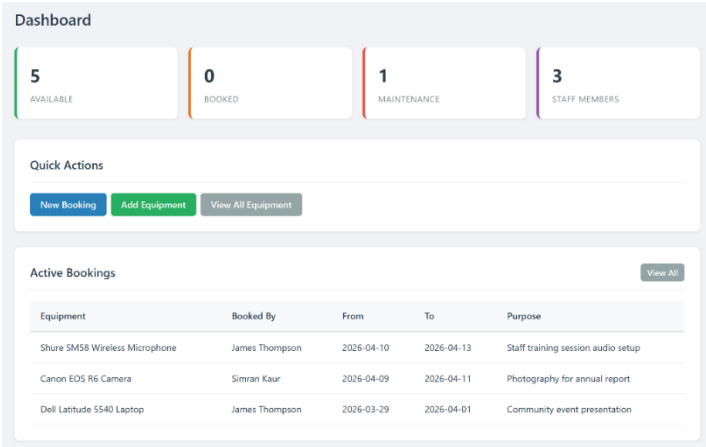
User Interface Implementation

The user interface is built with **Thymeleaf** templates served by five Spring MVC controllers. The layout uses a fragment-based approach: a shared fragments/layout.html file provides the navigation bar and footer, which are included in every page using Thymeleaf's th:replace directive. A single CSS stylesheet handles all visual styling.

The controllers follow a consistent pattern: each entity type (Category, User, Equipment, Booking) has its own controller with list, form and delete endpoints. The DashboardController aggregates statistics from all services to provide an overview page.

Controller	Routes	Features
DashboardController	GET /	Equipment stats, active bookings overview, quick action buttons
CategoryController	GET, POST /categories	CRUD with BR-06 deletion protection
UserController	GET, POST /users	CRUD with active booking deletion check
EquipmentController	GET, POST /equipment	CRUD with search by name, filter by category and status
BookingController	GET, POST /bookings	Create and cancel bookings with business rule validation

The interface is responsive, adapting to different screen sizes through CSS media queries. Status values are displayed using colour-coded badges (green for Available, amber for Booked, red for Maintenance). Forms include HTML5 validation attributes (required, type="date", type="email") as a first line of defence, with server-side Bean Validation as the authoritative check.



Categories

Add Category

ID	Name	Description	Equipment Count	Actions
1	Audio Equipment	Microphones, speakers, mixers and audio accessories	2	Edit Delete
2	Computing	Laptops, tablets and desktop workstations	2	Edit Delete
3	Photography	Cameras, lenses, tripods and lighting kits	2	Edit Delete

Users

Add User

ID	Name	Username	Role	Department	Email	Actions
1	Helen Murray	admin	ADMIN	Administration	h.murray@organisation.co.uk	Edit Delete
2	James Thompson	jthompson	STAFF	Events	j.thompson@organisation.co.uk	Edit Delete
3	Simran Kaur	skaur	STAFF	Training	s.kaur@organisation.co.uk	Edit Delete

Equipment

Add Equipment

SEARCH BY NAME: Equipment name...

CATEGORY: All categories

STATUS: All statuses

Filter Clear

ID	Name	Category	Location	Status	Cost/Day	Actions
1	Shure SM58 Wireless Microphone	Audio Equipment	Storage Room A	AVAILABLE	£12.00	Edit Delete
2	Yamaha MG12 Mixing Console	Audio Equipment	Storage Room A	AVAILABLE	£25.00	Edit Delete
3	Dell Latitude 5540 Laptop	Computing	IT Office	AVAILABLE	£18.00	Edit Delete
4	Lenovo ThinkPad T14 Laptop	Computing	IT Office	MAINTENANCE	£15.00	Edit Delete
5	Canon EOS R6 Camera	Photography	Storage Room B	AVAILABLE	£35.00	Edit Delete
6	Manfrotto 055 Tripod	Photography	Storage Room B	AVAILABLE	£8.00	Edit Delete

Bookings

New Booking

ID	Equipment	Booked By	From	To	Days	Purpose	Status	Actions
1	Shure SM58 Wireless Microphone	James Thompson	2026-04-10	2026-04-13	4	Staff training session audio setup	CONFIRMED	Cancel
2	Canon EOS R6 Camera	Simran Kaur	2026-04-09	2026-04-11	3	Photography for annual report	CONFIRMED	Cancel
3	Dell Latitude 5540 Laptop	James Thompson	2026-03-29	2026-04-01	4	Community event presentation	CONFIRMED	Cancel
4	Yamaha MG12 Mixing Console	Simran Kaur	2026-04-13	2026-04-13	1	Workshop recording	CANCELLED	

Unfamiliar Technologies and Constructs

This project required me to learn several technologies that were not covered in the HND course modules. The table below summarises each one, why I chose it, and what I gained from using it.

Technology	Why Selected	What I Learned
Spring Boot 4.0	Industry-standard framework for Java web applications. Provides dependency injection, auto-configuration and an embedded web server.	Constructor injection, @Service/@Controller annotations, application.properties configuration, layered architecture.
Spring Data JPA	Eliminates boilerplate data access code. Repository interfaces generate SQL from method names.	JPA entity mapping, derived query methods, custom JPQL queries, @DataJpaTest for repository testing.
Thymeleaf	Server-side templating that produces natural HTML. Integrates tightly with Spring MVC and supports fragments.	Template fragments for layout reuse, th:each iteration, th:if conditionals, form binding with th:object and th:field.
Bean Validation	Declarative constraint annotations on entity fields, validated automatically by Spring before processing.	@NotBlank, @Size, @Email annotations. BindingResult for capturing validation errors and returning them to forms.
Mockito	Mocking framework for isolating service-layer unit tests from database dependencies.	@Mock and @InjectMocks annotations, when/thenReturn stubbing, verify() for interaction checks, doThrow() for exception simulation.
Lombok	Reduces boilerplate by generating getters, setters and constructors at compile time.	@Getter, @Setter, @NoArgsConstructor. Requires annotation processing enabled in the IDE.

All of these were introduced through self-research using the official documentation, Baeldung tutorials and Stack Overflow. The Information Sources table in the Planning Stage lists the specific references I used.

Error Handling and Validation

Error handling is implemented at three levels: input validation on entity fields, business rule enforcement in the service layer, and centralised exception handling via a global controller advice.

Input Validation

Entity classes use **Bean Validation** annotations (@NotBlank, @NotNull, @Size, @Email, @DecimalMin) to enforce field-level constraints. When a form is submitted, the controller uses @Valid together with BindingResult to check for violations. If any are found, the form is redisplayed with error messages next to the offending fields rather than redirecting to an error page.

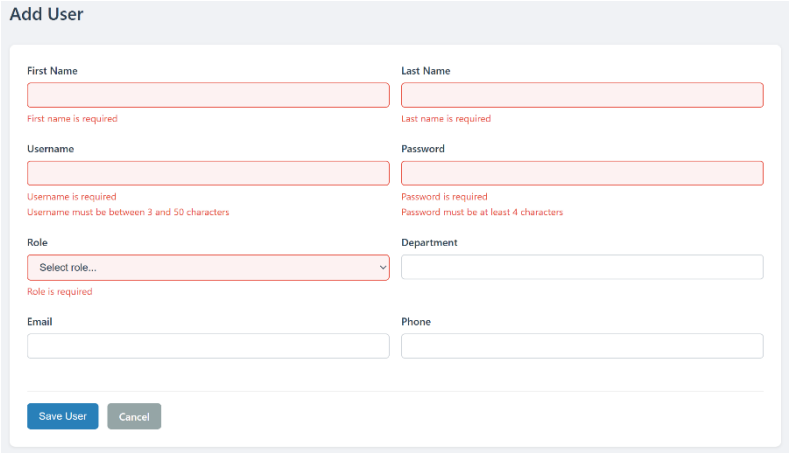
Global Exception Handler

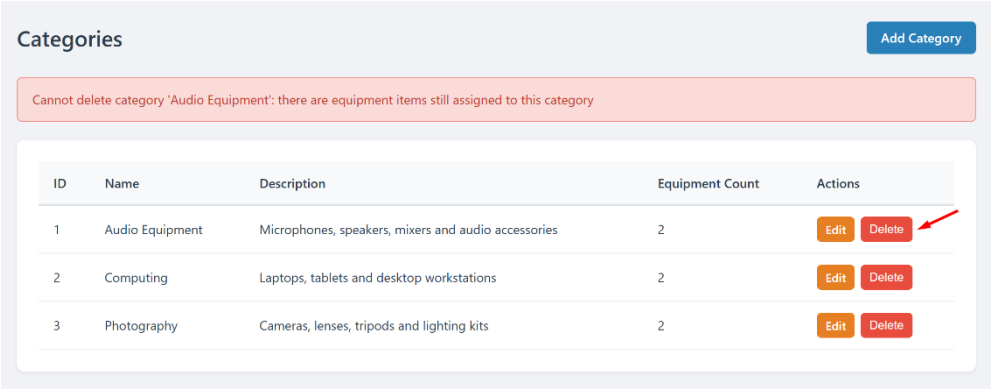
A @ControllerAdvice class (GlobalExceptionHandler) intercepts all unhandled exceptions and routes them to the error page with an appropriate status code and message. This ensures that users never see a raw stack trace. Each custom exception type has its own handler method, and a catch-all handler covers unexpected errors.

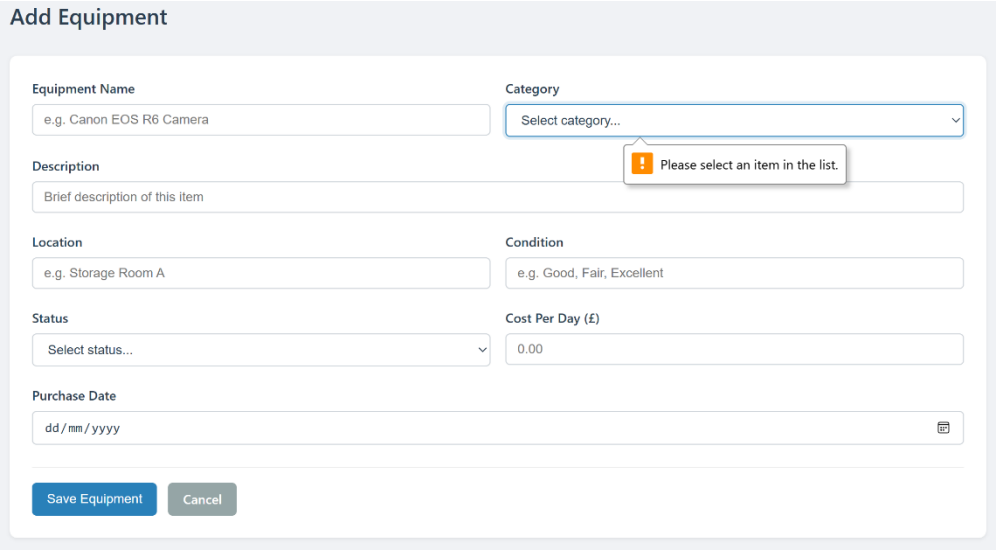
Application Logging

All services use SLF4J logging to record significant events. Successful operations (save, delete, status change) are logged at INFO level. Rejected actions (deletion blocked, booking conflict, equipment unavailable) are logged at WARN level.

Unexpected exceptions are logged at ERROR level with the full stack trace. This provides an audit trail for debugging without exposing internal details to the user.







Internal Documentation

The codebase follows consistent conventions for readability and maintainability:

Naming conventions

Classes use PascalCase (BookingService, EquipmentController). Methods and variables use camelCase with descriptive names (findConflictingBookings, verifyAvailableForBooking). Even loop variables and lambda parameters have meaningful names rather than single letters. Constants are not used in this project as there are no magic numbers; all configuration sits in application.properties.

Package structure

The project is organised into five packages: model (entities and enums), repository (data access interfaces), service (business logic), controller (request handling), config (application setup including DataInitialiser), and exception (custom exception types). Each package has a single clear responsibility.

Code comments

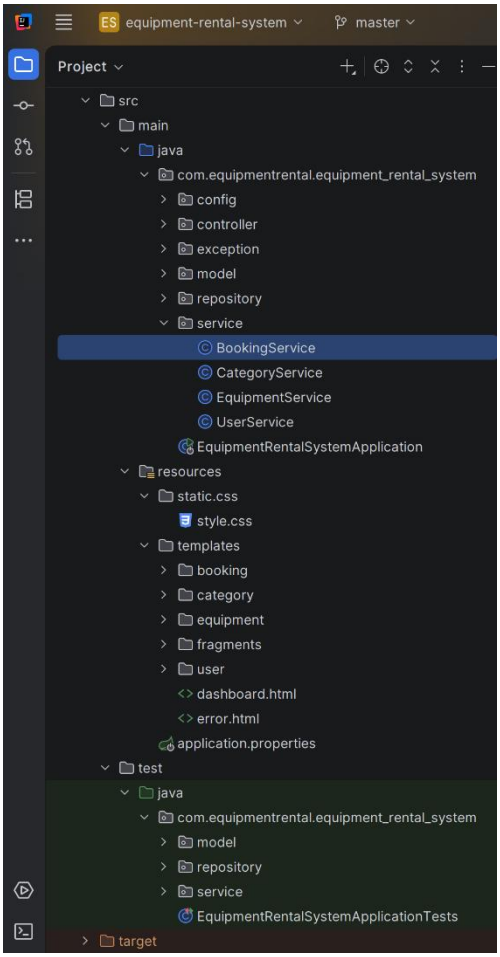
Comments are used where the intent is not obvious from the code itself. Business rule references (BR-01, BR-02, etc.) are noted in Javadoc on the methods that enforce them. The deliberate choice was to keep comments minimal: a well-named method like verifyAvailableForBooking() does not need a comment explaining that it checks availability.

Indentation and formatting

All code uses 4-space indentation, consistent with Java conventions and the IntelliJ IDEA default formatter. Braces follow the K&R style (opening brace on the same line). Blank lines separate logical blocks within methods.

EquipmentRentalSystemApplication.java | BookingService.java | Equipment.java | Booking.java | Category.java

```java
17  public class BookingService {
41
42      public List<Booking> findByUserId(Long userId) { return bookingRepository.findByUserId(userId); }
45
46      public List<Booking> findByEquipmentId(Long equipmentId) {  no usages  Stanislav Starishko
47          return bookingRepository.findByEquipmentId(equipmentId);
48      }
49
50      /**
51       * Creates a new booking after validating all business rules.
52       *
53       * Checks performed in order:
54       * 1. BR-05: end date must be equal to or later than start date
55       * 2. BR-03: booking must reference a valid registered user
56       * 3. BR-01: equipment must not be in MAINTENANCE or UNAVAILABLE status
57       * 4. BR-02: no overlapping confirmed bookings for the same equipment
58       * 5. BR-08: equipment status updated to BOOKED after successful creation
59       */
60      @Transactional  12 usages  Stanislav Starishko
61      public Booking createBooking(Booking booking) {
62
63          // BR-05: validate date range
64          if (booking.getDateFrom().isAfter(booking.getDateTo())) {
65              log.warn("Booking rejected: invalid date range {} to {}",
66                      booking.getDateFrom(), booking.getDateTo());
67              throw new InvalidBookingDatesException(
68                      booking.getDateFrom().toString(),
69                      booking.getDateTo().toString());
70          }
71
72          // BR-03: verify user exists
73          User bookingUser = userService.findById(booking.getUser().getId());
74
75          // BR-01: verify equipment is available for booking
76          Equipment equipment = equipmentService.findById(booking.getEquipment().getId());
77          equipmentService.verifyAvailableForBooking(equipment);
78
79          // BR-02: check for conflicting bookings
80          List<Booking> conflictingBookings = bookingRepository.findConflictingBookings(
81                  equipment.getId(),
82                  booking.getDateFrom(),
```



Testing

This section covers the testing strategy, test plan, test execution and results. Testing was approached as a structured process: first the plan was defined, then automated tests were written and executed, followed by a set of manual browser-based scenarios. I included manual testing deliberately because automated unit tests verify individual components in isolation but cannot catch every issue that appears when the full application runs in the browser. If any defects are found during manual testing, they will be fixed, covered by a new regression test, and verified with a second automated test run.

Test Plan

Testing Approach

The testing strategy combines three methods, each targeting a different layer of the application:

Unit tests (JUnit 5 with Mockito) cover the service layer and entity domain methods. Services are tested in isolation: repository and service dependencies are mocked so that each test verifies only the business logic of the class under test. Entity tests use plain JUnit without Spring context, testing domain methods like date conflict detection, booking duration and status transitions.

Integration tests (JUnit 5 with @DataJpaTest) cover the repository layer. These run against a real H2 database to verify that custom JPQL queries return correct results. The Spring test context manages transaction rollback between tests automatically.

Manual tests cover end-to-end scenarios through the browser. These verify that controllers, templates, validation and business rules work together as the user would experience them. Manual testing is deliberately limited to a small number of high-value scenarios that exercise paths not easily covered by unit tests alone.

Tools

Tool	Purpose
JUnit 5	Test framework with @Nested grouping and @DisplayName annotations
Mockito	Mocking dependencies for service-layer isolation
@DataJpaTest	Spring Boot test slice for repository integration tests with H2
IntelliJ IDEA	Test runner and HTML report generation

Test Cases

The table below lists all automated test cases grouped by class. Each group corresponds to a @Nested inner class in the test code. The naming convention follows the pattern "Should..." for consistency and readability.

BookingTest (15 tests) — entity domain methods, no Spring context

Group	Tests	What Is Verified
Date Conflict Detection	9	hasConflictWith() for full overlap, partial overlap, boundary dates, same-day, no overlap, symmetry
Booking Status	3	isActive() returns true for CONFIRMED, false for CANCELLED; cancel() sets status correctly
Duration Calculation	3	getDurationDays() for same-day, two-day and multi-day bookings

BookingServiceTest (16 tests) — service layer with Mockito mocks

Group	Tests	What Is Verified
Create Booking	2	Successful creation with CONFIRMED status; BR-08 equipment status updated to BOOKED
Date Validation (BR-05)	2	Rejects end date before start date; accepts same-day booking
Equipment Availability (BR-01)	4	Rejects MAINTENANCE and UNAVAILABLE; allows AVAILABLE and BOOKED (if no date conflict)
Booking Conflict (BR-02)	2	Rejects when conflicting booking exists; allows when no conflict
User Validation (BR-03)	1	Rejects booking for non-existent user
Cancel Booking	2	Sets status to CANCELLED; throws exception for non-existent booking
Find Booking	3	Find by ID, not found throws exception, find by user ID

EquipmentServiceTest (14 tests) — service layer with Mockito mocks

Group	Tests	What Is Verified
Availability Verification (BR-01)	4	Allows AVAILABLE and BOOKED; rejects MAINTENANCE and UNAVAILABLE
Deletion (BR-07)	4	Allows with no bookings or only cancelled; blocks with active bookings; throws for non-existent
Status Update (BR-08)	2	Updates to BOOKED; updates to AVAILABLE
Search and Filter	4	Find by category, by status, search by name, empty results

CategoryServiceTest (7 tests) — service layer with Mockito mocks

Group	Tests	What Is Verified
Find Category	3	Find by ID, not found throws exception, find all
Save Category	1	Save and return with generated ID
Deletion (BR-06)	3	Allows with no equipment; blocks with assigned equipment; throws for non-existent

BookingRepositoryTest (10 tests) — integration tests with @DataJpaTest

Group	Tests	What Is Verified
Conflict Detection (BR-02)	7	JPQL query: full overlap, partial overlap, boundary date, no overlap, ignores cancelled, different equipment, multiple conflicts
Query Methods	3	findByEquipmentId, findByUserId, findByStatus

Total automated tests: 62

Manual Test Scenarios

The following scenarios are tested through the browser after all automated tests pass. Each scenario exercises an end-to-end workflow that depends on the interaction between multiple layers.

ID	Scenario	Steps	Expected Result
MT-01	Create a valid booking	Navigate to New Booking, select available equipment, enter valid dates, submit	Booking appears in list with CONFIRMED status
MT-02	Attempt double-booking	Create a booking for an item, then try to book the same item for overlapping dates	Error message about booking conflict

ID	Scenario	Steps	Expected Result
MT-03	Book equipment under maintenance	Set equipment status to MAINTENANCE, attempt to book it	Error message about equipment not available
MT-04	Cancel a booking and check equipment status	Cancel the only active booking for an item, check equipment status on Equipment page	Equipment status should return to AVAILABLE
MT-05	Delete category with equipment	Attempt to delete a category that has equipment assigned	Error message about assigned equipment
MT-06	Submit form with empty required fields	Open Add User form, leave all fields blank, click Save	Validation errors displayed next to each required field

Test Run 1: Automated Tests

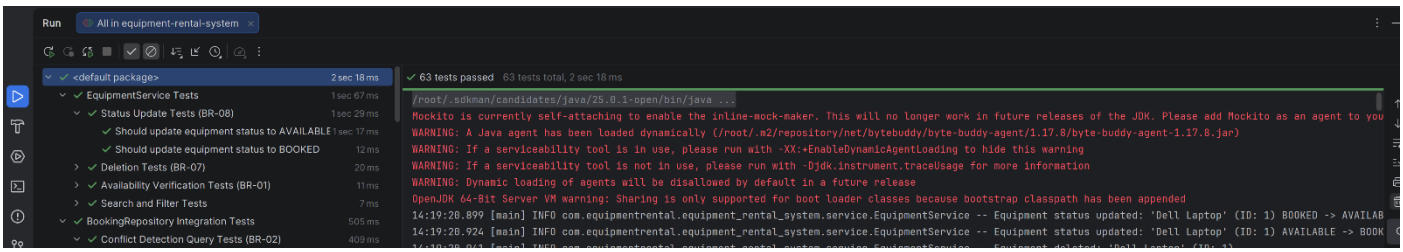
The first test run was executed in IntelliJ IDEA after completing all development phases (1 through 5). All 63 tests passed successfully on the first attempt.

Results summary:

Metric	Value
Total tests	63
Passed	63
Failed	0
Duration	2 sec 18 ms

The breakdown by test class:

Test Class	Tests	Result
BookingTest	15	All passed
BookingServiceTest	16	All passed
EquipmentServiceTest	14	All passed
CategoryServiceTest	7	All passed
BookingRepositoryTest	10	All passed
EquipmentRentalSystemApplicationTests	1	Passed



The full HTML report is saved as **docs/test_results/Test_Results_All_in_equipment-rental-system_01.html**.

All in equipment-rental-system: 63 total, 63 passed 2.02 s

Collapse | Expand

EquipmentService Tests 1.07 s

Status Update Tests (BR-08) 1.03 s

Should update equipment status to AVAILABLE passed 1.02 s

```
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (/root/.m2/repository/net/bytebuddy/byte-buddy-agent/1.17.8/byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.Instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
14:19:20.899 [main] INFO com.equipmentrental.equipment_rental_system.service.EquipmentService -- Equipment status updated: 'Dell Laptop' (ID: 1) BOOKED -> AVAILABLE
```

Should update equipment status to BOOKED passed 12 ms

```
14:19:20.924 [main] INFO com.equipmentrental.equipment_rental_system.service.EquipmentService -- Equipment status updated: 'Dell Laptop' (ID: 1) AVAILABLE -> BOOKED
```

Deletion Tests (BR-07) 20 ms

Should delete equipment with no active bookings passed 6 ms

```
14:19:20.941 [main] INFO com.equipmentrental.equipment_rental_system.service.EquipmentService -- Equipment deleted: 'Dell Laptop' (ID: 1)
```

Should block deletion when active bookings exist passed 6 ms

```
14:19:20.947 [main] WARN com.equipmentrental.equipment_rental_system.service.EquipmentService -- Deletion blocked for equipment 'Dell Laptop' (ID: 1): active bookings exist
```

Should delete equipment with no bookings at all passed 5 ms

```
14:19:20.954 [main] INFO com.equipmentrental.equipment_rental_system.service.EquipmentService -- Equipment deleted: 'Dell Laptop' (ID: 1)
```

Should throw exception when deleting non-existent equipment passed 3 ms

Availability Verification Tests (BR-01) 11 ms

Should allow booking for equipment with BOOKED status passed 2 ms

Should reject booking for unavailable equipment passed 4 ms

```
14:19:20.968 [main] WARN com.equipmentrental.equipment_rental_system.service.EquipmentService -- Booking rejected: equipment 'Dell Laptop' (ID: 1) has status UNAVAILABLE
```

Should allow booking for equipment with AVAILABLE status passed 2 ms

Should reject booking for equipment under maintenance passed 3 ms

```
14:19:20.976 [main] WARN com.equipmentrental.equipment_rental_system.service.EquipmentService -- Booking rejected: equipment 'Dell Laptop' (ID: 1) has status MAINTENANCE
```

Search and Filter Tests 7 ms

At this point all automated tests confirmed that the business rules, service layer and repository queries are working as expected. The next step was manual testing to verify the end-to-end behaviour through the browser.

Manual Testing

After the automated test run, I performed a set of manual browser-based tests to verify end-to-end behaviour. The table below documents each scenario with its expected result, actual result and any follow-up actions required.

ID	Scenario	Steps	Expected Result	Actual Result	Status	Comment
MT-01	Create a valid booking	Navigate to New Booking, select available equipment, enter valid dates, submit	Booking appears in list with CONFIRMED status	Booking created (ID 5, Manfrotto 055 Tripod, 09/04 to 15/04, CONFIRMED). Success message displayed.	Pass	As expected
MT-02	Attempt double-booking	Create a booking for an item, then try to book the same item for overlapping dates	Error message about booking conflict	Conflict message displayed correctly. However, the form is cleared after the error, losing all user input.	Pass (with UX issue)	BR-02 works correctly. Form data loss on error to be fixed.
MT-03	Book equipment under maintenance	Set equipment status to MAINTENANCE, attempt to book it	The UI prevents invalid selection rather than showing an error after submission.	Equipment with MAINTENANCE status is displayed in the dropdown but disabled and cannot be selected. Booking for unavailable equipment is prevented at the UI level.	Pass	As expected

ID	Scenario	Steps	Expected Result	Actual Result	Status	Comment
MT-04	Cancel booking and check equipment status	Cancel the only active booking for an item, check equipment status on Equipment page	Equipment status should return to AVAILABLE	Booking was cancelled successfully (status changed to CANCELLED). However, the equipment status remained **BOOKED** on the Equipment page instead of reverting to AVAILABLE.	Fail	BR-08 violation. The cancelBooking() method does not check whether other active bookings exist for the equipment and does not revert the status. To be fixed.
MT-05	Delete category with equipment	Attempt to delete a category that has equipment assigned	Error message about assigned equipment	Error message displayed: "Cannot delete category 'Audio Equipment': there are equipment items still assigned to this category."	Pass	As expected
MT-06	Submit form with empty required fields	Open Add User form, leave all fields blank, click Save	Validation errors displayed next to each required field	Validation error messages displayed next to each required field (first name, last name, username, password, role). Form retained empty state with errors visible.	Pass	As expected

Bug Fixes and Regression Testing

Manual testing revealed two issues that required code changes.

Issue 1: Form data loss on booking conflict (MT-02). When a booking was rejected due to a date conflict, the form was cleared and the user lost all entered data. This happened because the controller used a redirect on error, which creates a new GET request with an empty form.

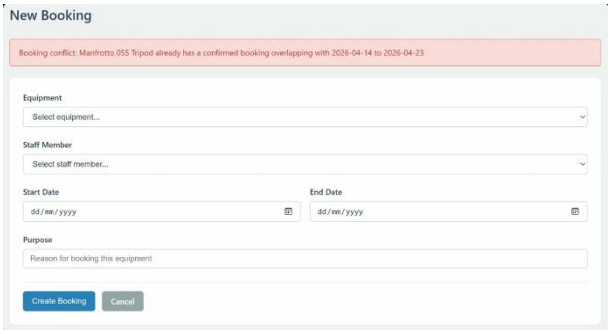
Fix applied. The BookingController.save() method was changed to return the form view directly on error instead of redirecting. The selected equipment ID and user ID are passed back to the model, and th:selected attributes were added to the dropdown elements in the template. The @DateTimeFormat annotation was added to the date fields in the Booking entity to ensure dates survive the round-trip. After the fix, all form fields (equipment, staff member, dates and purpose) are preserved when a validation or conflict error occurs.

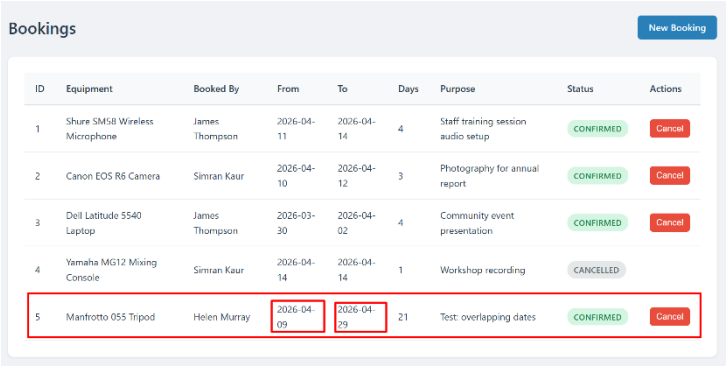
Issue 2: Equipment status not reverted after cancellation (MT-04). When the only active booking for an equipment item was cancelled, the equipment status remained **BOOKED** instead of reverting to **AVAILABLE**. This is a violation of **BR-08**, which states that equipment status must automatically update to reflect active bookings.

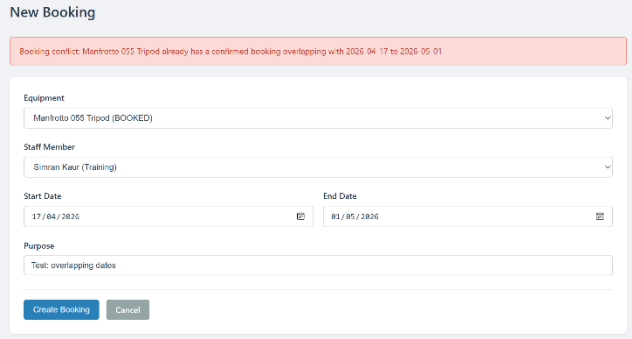
Root cause. The cancelBooking() method in BookingService set the booking status to CANCELLED and saved it, but did not check whether any other active bookings still existed for that equipment. The status update logic was present in createBooking() (setting status to BOOKED) but the reverse operation was missing from the cancellation path.

Fix applied. After cancelling the booking, the method now queries the repository for any remaining active (CONFIRMED) bookings for the same equipment item. If none exist, it calls equipmentService.updateStatus() to revert the status to AVAILABLE. If other active bookings remain, the status stays as BOOKED.

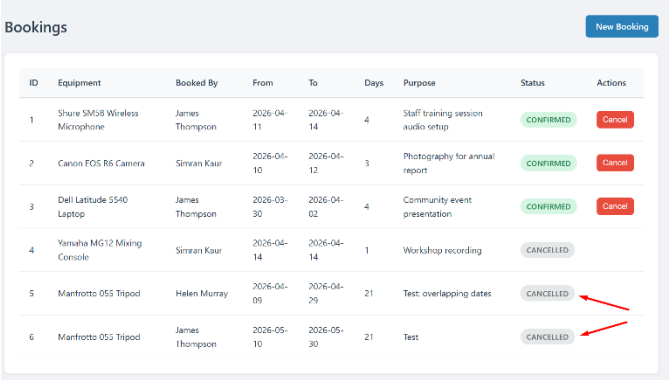
MT-02 screenshots:

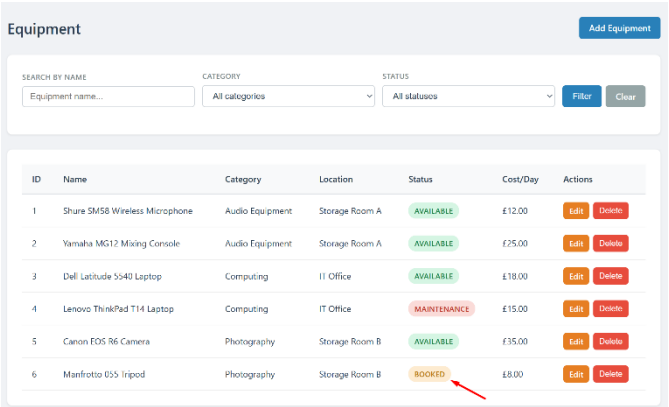


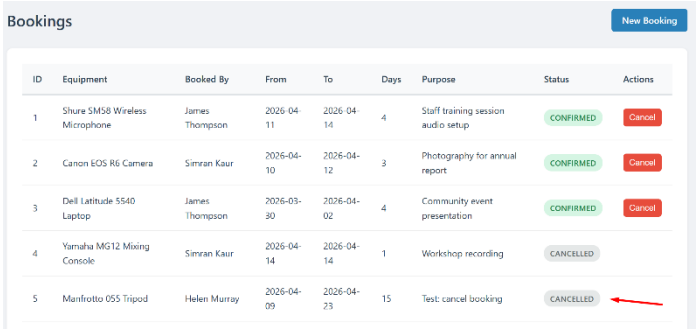


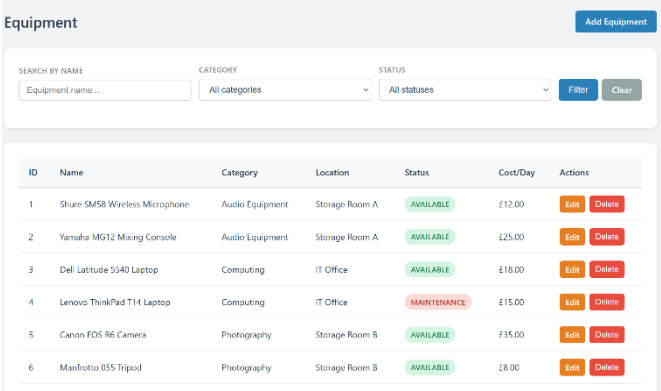


MT-04 screenshots:









Regression test added. A new unit test was added to BookingServiceTest in the Cancel Booking group:

- **"Should revert equipment status to AVAILABLE when no other active bookings remain (BR-08)"** - mocks a single active booking, cancels it, and verifies that equipmentService.updateStatus() is called with AVAILABLE.

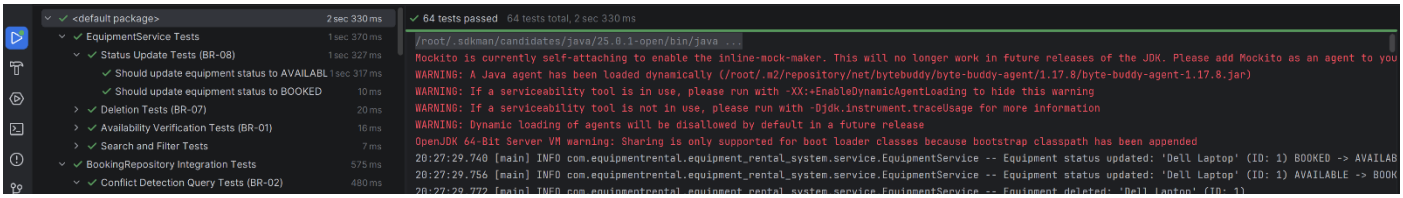
This brings the total automated test count from 63 to **64**.

Test Run 2: After Bug Fix

After applying the fix and adding the regression test, all automated tests were executed again.

Results summary:

Metric	Value
Total tests	64
Passed	64
Failed	0
Duration	2 sec 330 ms

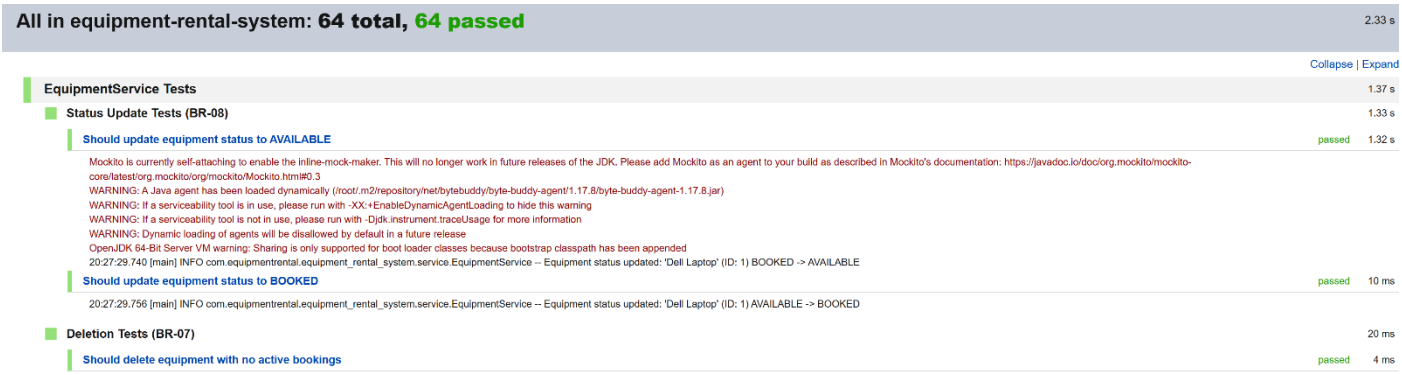


The breakdown by test class:

Test Class	Tests	Result
BookingTest	15	All passed
BookingServiceTest	17 (+1 regression test)	All passed
EquipmentServiceTest	14	All passed
CategoryServiceTest	7	All passed
BookingRepositoryTest	10	All passed
EquipmentRentalSystemApplicationTests	1	Passed

The new regression test confirmed that the fix works correctly: cancelling the only active booking for an equipment item now reverts its status to AVAILABLE. All previously passing tests remained green, confirming that the fix introduced no regressions.

The full HTML report is saved as **docs/test_results/Test_Results_All_in_equipment-rental-system_02.html**.



User Documentation

Approach

When I reached this phase, I considered two options: writing a separate PDF user manual, or building help features directly into the application. I chose the second approach for a practical reason: a standalone document gets separated from the application and becomes outdated the moment the interface changes. Built-in help stays with the product and is accessible exactly where the user needs it.

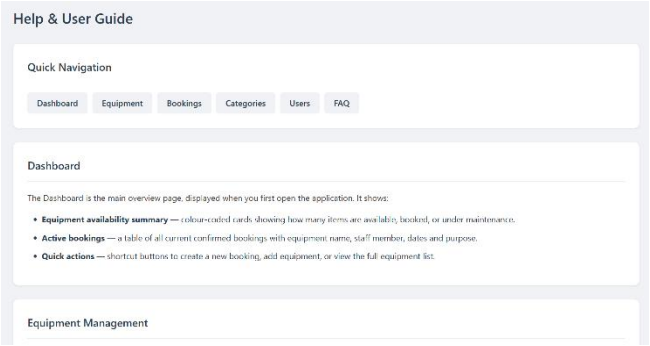
This decision was made during the testing phase (Phase 6), when manual testing highlighted areas where a user would benefit from contextual guidance. For example, the booking form has specific rules about date ranges and equipment availability that are not obvious from the form layout alone. Rather than expecting users to consult an external document, the application now explains these rules in context.

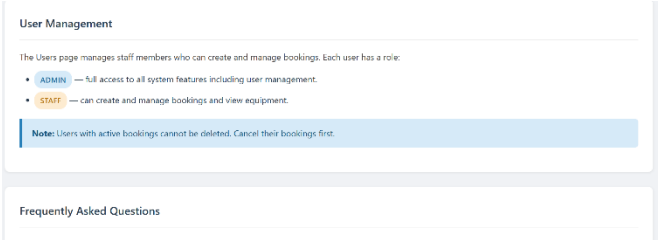
Help Page

The application includes a dedicated **Help & User Guide** page, accessible via the "Help" link in the main navigation bar. The page is structured into sections that mirror the application's main areas: Dashboard, Equipment Management, Booking Management, Category Management and User Management.

Each section explains what the page does, how to use its key features, and what restrictions apply. For example, the Bookings section provides a numbered step-by-step guide for creating a booking and explains what happens when a booking is cancelled.

The page includes anchor-based navigation at the top, allowing users to jump directly to the relevant section without scrolling.





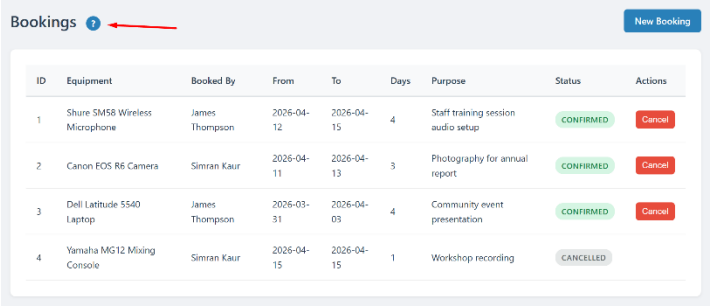
Contextual Help

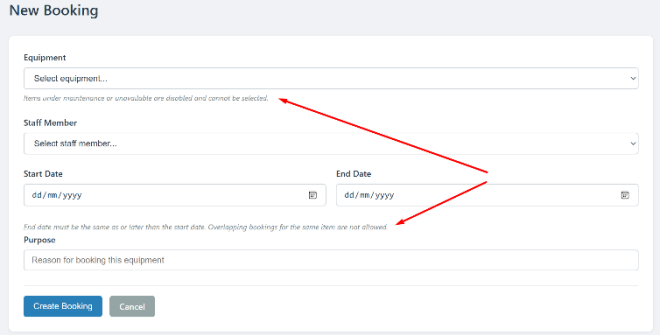
In addition to the central help page, each main page in the application includes a **contextual help link** ("?" icon) next to the page title. Clicking this link takes the user directly to the corresponding section of the help page. For example, clicking "?" on the Equipment page opens /help#equipment.

Key forms also include **inline hints** below specific fields:

- **Booking form:** "Items under maintenance or unavailable are disabled and cannot be selected" appears below the equipment dropdown. "End date must be the same as or later than the start date. Overlapping bookings for the same item are not allowed" appears below the date fields.
- **Equipment form:** "Setting status to Maintenance or Unavailable will prevent new bookings for this item" appears below the status dropdown.

These hints address the most common points of confusion identified during manual testing, without cluttering the interface.





FAQ Section

The help page includes a **Frequently Asked Questions** section at the bottom, covering six common scenarios:

- Can I book equipment that is already booked?
- Why can I not select some equipment in the booking form?
- What happens to the equipment status when I cancel a booking?
- Why can I not delete a category?
- Why can I not delete a user?
- What does the cost per day field mean?

Each answer is written in plain language and directly addresses the business rule behind the behaviour. The questions were selected based on the scenarios that came up during manual testing and the business rules that are most likely to surprise a first-time user.

After implementing the help features, all 64 automated tests were re-run to confirm that the changes introduced no regressions.

Technical Documentation (Javadoc)

In addition to the user-facing help features, I produced technical documentation aimed at developers who might maintain or extend the system in the future. This was done using **Javadoc**, Java's built-in documentation generation tool.

I added Javadoc comments to all public classes and methods after completing the testing phase and confirming that the application was stable. This was a deliberate decision: documenting code that is still being debugged or refactored would create maintenance overhead and risk outdated descriptions. Once the second test run confirmed all 64 tests passing and both manual testing issues resolved, I went through every entity, enum, repository, service, exception and controller class, adding structured comments with @param, @return, @throws and @see tags.

The comments focus on what each class does, which business rules it enforces, and how it relates to other parts of the system. For example, the BookingService class documentation lists all five business rules it validates (BR-01, BR-02, BR-03, BR-05, BR-08) and links to the specific methods in EquipmentService and UserService that it delegates to.

The HTML documentation was generated using the Maven Javadoc plugin (./mvnw javadoc:javadoc) and is available in docs/apidocs/. It provides a navigable reference with package summaries, class hierarchies, method signatures and cross-references between related classes.

Conclusion

This portfolio documents the full Development Stage of the Equipment and Resource Booking Management System. All eight phases of the development roadmap were completed: from initial project setup through to testing, bug resolution, user documentation and code review.

The finished application implements all eight business rules defined in the Planning Stage (BR-01 to BR-08), with each rule enforced in the appropriate service layer and verified by automated tests. The final codebase comprises 4 JPA entities, 3 enums, 4 repositories, 4 services, 5 controllers, 5 custom exceptions, a global exception handler, 11 Thymeleaf templates and a data initialiser.

Testing produced 64 automated tests across 5 test classes, covering entity domain logic, service-layer business rules and repository query behaviour. Two issues were identified during manual testing: a UX problem with form data loss on booking conflicts (MT-02) and a BR-08 violation where equipment status was not reverted after booking cancellation (MT-04). Both were fixed and verified with a second test run.

The application includes two levels of documentation aimed at different audiences. For end users, a **Help and User Guide** page is built directly into the application, with contextual hints on forms and a FAQ section covering common scenarios. For technical users and future developers, **Javadoc** documentation was generated using Java's built-in documentation tool. Javadoc comments were added to all public classes and methods after the application was confirmed stable through testing, to avoid documenting code that might still change. The generated reference is available in docs/apidocs/.

Two adjustments were made to the original plan during development. Phase 7 was renamed from "User Documentation & Help" to "UI Refinement and User Help" to reflect the decision to build help features into the application rather than producing a separate manual. The booking form was also improved during testing to retain user input when a validation error occurs. Both changes were driven by practical findings during testing and improved the overall quality of the product.

The application is complete, tested and documented, and is ready for the **Evaluation Stage**.